Nginx Config Assignment

Study how nginx works and the workflow when the when we write the server name in browser

1.Type the host name localhost.com on our browser

2. The browser asks the OS of our host about the ip address associated with the server name.

3. OS has a special built in rule where localhost::: 127.0.0.1 (loopback address)

4. Computer Sends the request packet not to internet but internally to 127.0.0.1 on port 80

5. Nginx listens on port 80

6. Nginx looks at the host header in its config, finds the exact virtual server block and location of html file

7. Nginx reads the content of index.html file and combine it with the responsive code

8. The response code is sent internally back to our browser over the loopback connection
Browser renders the received the html file

Try to give a custom server name and see what all configs are changing for that. Are we able to share the custom server link with another person?

1.Made a directory on our local host and added an html file

```HTML
<!DOCTYPE html>
<html>
<head>
    <title>My Local App</title>
</head>
<body>
    <h1>Success! This is myapp.local served by NGINX in Multipass.</h1>
</body>
</html>
```

2. Launched a vm and transferred file to that vm

3. Create a custom nginx file and add the custom server name as `myapp.local:`

```
sudo ln -s /etc/nginx/sites-available/myapp.local.conf
/etc/nginx/sites-enabled/
```

```HTML
server {
    listen 80;
    # 1. NEW SERVER NAME
    server_name myapp.local www.myapp.local;
    # 2. POINTS TO THE MOUNTED DIRECTORY
    root /home/ubuntu/myapp;
    index index.html;
    location / {
        try_files $uri $uri/ =404;
    }
}
```

4. Removed the custom nginx html file - `sudo rm /etc/nginx/sites-enabled/default`

5. sudo systemctl reload nginx

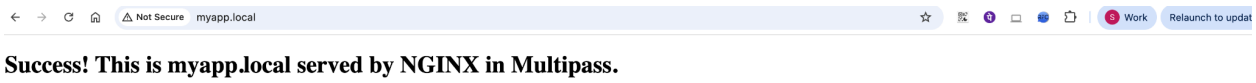
6. In etc/nginx/nginx.conf added the user as ubuntu to allow access to nginx for the html file

```
[ubuntu@nginx-server:/etc/nginx$ cat nginx.conf
user ubuntu;
worker_processes auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
        worker_connections 768;
        # multi_accept on;
}

http {
```

7. Exit from vm and add the ip address and server name in the /etc/host file

```
##
# Host Database
#
# localhost is used to configure the loopback interface
# when the system is booting.  Do not change this entry.
##
127.0.0.1       localhost
255.255.255.255 broadcasthost
::1             localhost
192.168.64.57    myapp.local
```

8. Check on the browser if our custom server name is rendered



Success! This is myapp.local served by NGINX in Multipass.

No we cannot render this custom server on some other machine because that machine will not have any mapping of our server name and its on its etc/hosts config

What is site-available and site enabled?

`sites-available` (The Storage)

- This directory contains all the configuration files you have written for every website you *could* host.
- The files here are **not active**. NGINX does not read this folder when it starts up.
- **Purpose:** It allows you to work on new configurations, keep backups, or store "drafts" of site settings without them affecting your live server.

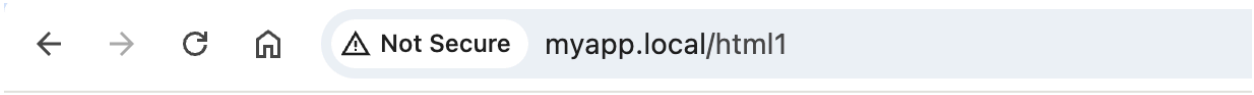
`sites-enabled` (The Active Switch)

- This directory contains **symbolic links** (shortcuts) that point back to the actual files in `sites-available`.
- NGINX is configured (via the main `nginx.conf`) to read every file in this folder when it starts or reloads.

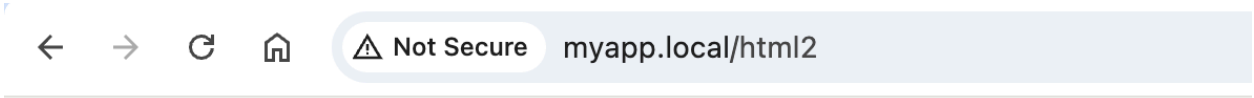
- **Purpose:** To "activate" a site, you create a link here. To "deactivate" a site (take it offline), you simply delete the link. The original configuration remains safe in `sites-available`.

Create a custom server name and under those define three html files. Check the browser for ip/html1,ip/html2, ip/html3

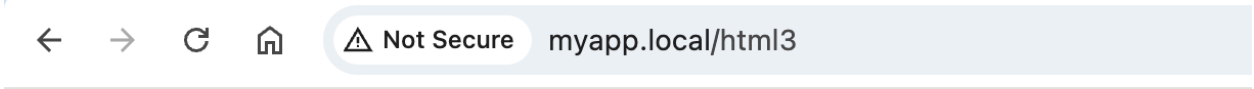
```
ubuntu@nginx-server:~$ cd myapp
ubuntu@nginx-server:~/myapp$ ls
index.html
ubuntu@nginx-server:~/myapp$ echo "<h1>Page 1</h1>" > html1.html
ubuntu@nginx-server:~/myapp$ echo "<h1>Page 2</h1>" > html2.html
ubuntu@nginx-server:~/myapp$ echo "<h1>Page 3</h1>" > html3.html
ubuntu@nginx-server:~/myapp$ cd /etc/nginx/sites-enabled
ubuntu@nginx-server:/etc/nginx/sites-enabled$ ls
myapp.local.conf
ubuntu@nginx-server:/etc/nginx/sites-enabled$ vim myapp.local.conf
ubuntu@nginx-server:/etc/nginx/sites-enabled$ sudo vim myapp.local.conf
ubuntu@nginx-server:/etc/nginx/sites-enabled$ sudo systemctl reload nginx
ubuntu@nginx-server:/etc/nginx/sites-enabled$ ls
myapp.local.conf
ubuntu@nginx-server:/etc/nginx/sites-enabled$ cd .
ubuntu@nginx-server:/etc/nginx/sites-enabled$ cd..
cd..: command not found
ubuntu@nginx-server:/etc/nginx/sites-enabled$ cd ..
ubuntu@nginx-server:/etc/nginx$ ls
conf.d  fastcgi.conf  fastcgi_params  koi-utf  koi-win  mime.types  modules-available  modules-enabled  nginx.conf  proxy_params  scgi_params  sites-available  sites-enabled  snippets  uwsgi_params  win-utf
ubuntu@nginx-server:/etc/nginx$ cd
ubuntu@nginx-server:~$ cd myapp
ubuntu@nginx-server:~/myapp$ ls
html1.html  html2.html  html3.html  index.html
ubuntu@nginx-server:~/myapp$
```



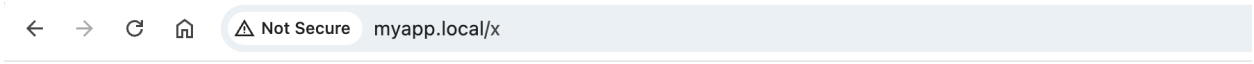
Page 1



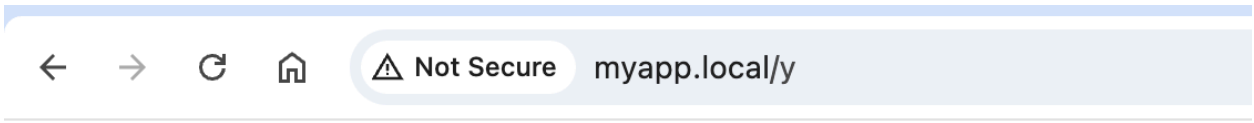
Page 2



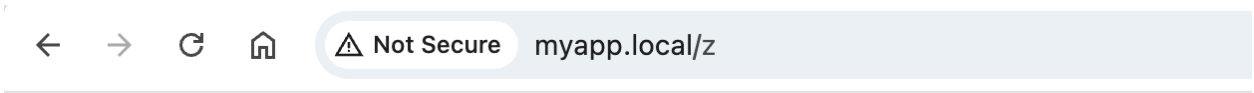
Page 3



Page 1



Page 2



Page 3

```
ubuntu@nginx-server:/etc/nginx/sites-available$ cat myapp.local.conf
server {
    listen 80;
    server_name myapp.local www.myapp.local;
    root /home/ubuntu/myapp;
    index index.html;

    location / {
        try_files $uri $uri.html $uri/ =404;
    }
}
```